PROGRAMMING IN JAVA — ASSIGNMENT 2 (CO2)

Name: J.Subhaashree Register Number: 192424008

Basis of report

This report is based on the uploaded executed Google Colab notebook. Recorded values are runtime-specific. Q2 notebook output displayed only the first 15 rows (head(15)); the included Q2 CSV is therefore explicitly a preview and must be replaced with the full CSV downloaded from the student's Colab session.

Q1 — String construction

Workload	String + (ns)	StringBuilder (ns)	StringBuffer (ns)
1x10	40,306	16,203	18,659
1000x50	19,896,378	10,277,660	12,653,661
3000x100	44,643,465	20,972,140	24,562,781

String is immutable; repeated concatenation creates intermediate strings. StringBuilder mutates a buffer and was fastest in the three measured workloads. StringBuffer is synchronized and may add overhead. Repeated concatenation can require $O(L^2)$ character copying, while StringBuilder is amortized O(L) in final output length.

Q2 — List collections

The experiment compares add-at-end, add-at-front, middle insert/remove, indexed access and iteration for ArrayList, LinkedList and ArrayDeque at sizes 1,000, 5,000 and 10,000. ArrayList provides O(1) indexed access and amortized O(1) append, but O(n) front/middle shifts. LinkedList has O(1) node insertion after locating the node, but O(n) indexed access/locating. ArrayDeque provides amortized O(1) operations at both ends and no indexed access. Q2 timings shown in the uploaded notebook are only a 15-row preview.

Q3 — hashCode and equals

The broken class retained two equal-registration objects (observed set size 2). The corrected class printed 'Unique students: 2'. HashSet uses hashCode to choose a bucket and equals to compare candidates. Equal objects must have equal hash codes; fields used by these methods should not be mutated while stored in a hash collection.

Q4 — Map comparison

Size	Map	100k lookups (ns)	Code report (ns)	Iteration report (ns)
1000	HashMap	16,287,205	3,864,748	1,171,406
1000	TreeMap	32,890,351	1,073,030	831,894
1000	LinkedHashMap	5,737,213	549,496	652,558
5000	HashMap	11,179,765	6,438,083	1,305,380
5000	TreeMap	32,095,100	1,413,207	2,373,270
5000	LinkedHashMap	8,298,110	729,181	1,083,756
10000	HashMap	8,865,000	10,154,766	4,740,020
10000	TreeMap	36,939,531	3,185,882	5,134,522
10000	LinkedHashMap	10,765,897	1,498,786	1,936,141

HashMap: average O(1) put/get with unspecified order. TreeMap: O(log n) and sorted keys. LinkedHashMap: average O(1) and insertion order. TreeMap suits code-order reports; LinkedHashMap suits order-added reports.

Q5 — Generic leaderboard

Output: [Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]. Ranking uses points descending, goal difference descending and name ascending. Bisons precedes Panthers because the first two fields tie and the name is alphabetically earlier. A generic Leaderboard accepts Comparator and sorts a copy in O(n log n).

Q6 — Emergency triage

Output: Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5). The comparator prioritizes the lower severity number and then earlier arrival counter. PriorityQueue add/poll are O(log n), peek is O(1), and space is O(n). Aging can reduce starvation of less severe patients.

Patients	Add all (ns)	Remove all (ns)
1000	1,598,531	3,187,201
5000	1,165,776	5,868,066
10000	2,631,866	9,905,697

Q7 — Diagnose, redesign and validate

ArrayList front insertion shifts existing references, making repeated front insertion $O(n^2)$ overall. The redesign uses ArrayDeque.addFirst(), amortized O(1) per insertion. The measured run supports the prediction at all four sizes; ArrayDeque was faster. At 20,000 insertions, ArrayList took 28,986,867 ns and ArrayDeque took 3,428,460 ns. ArrayDeque trades away indexed access, so ArrayList remains useful for random-access workloads.

Insertions	ArrayList (ns)	ArrayDeque (ns)
1000	240,772	168,337
5000	1,946,416	788,667
10000	7,483,076	1,472,004
20000	28,916,867	3,428,460

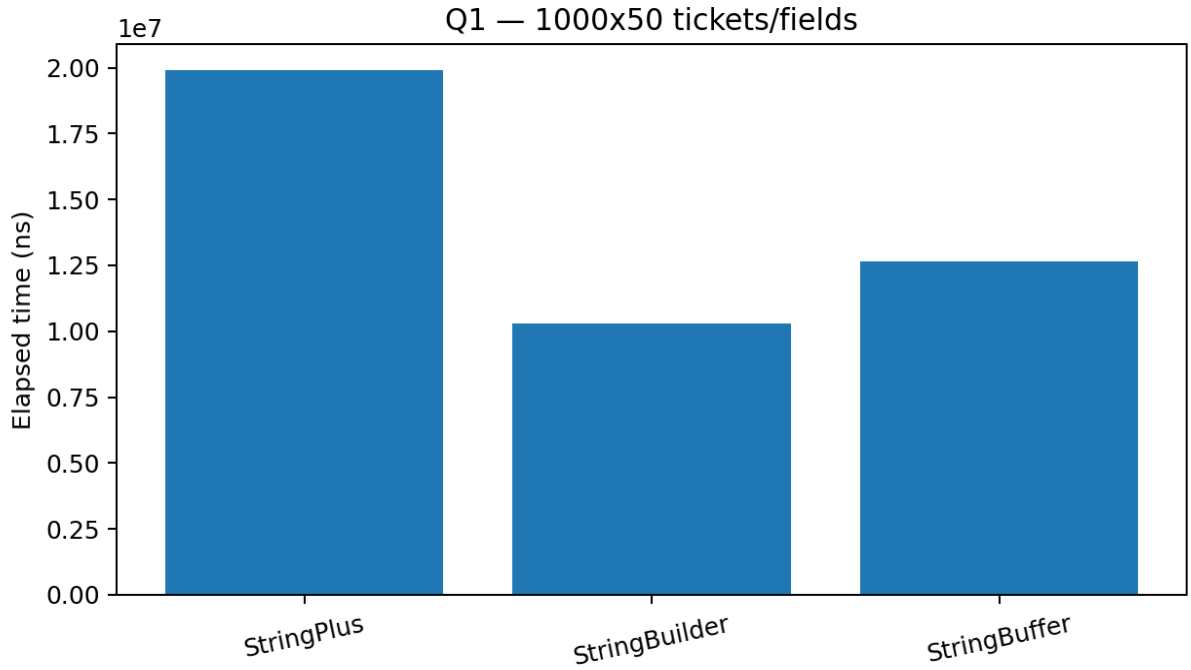
Conclusion

The measured experiments support choosing Java structures according to access patterns: StringBuilder for local text generation, ArrayList for indexed lists, ArrayDeque for double-ended queues, correctly implemented equals/hashCode for hash sets, TreeMap/LinkedHashMap for ordered reports, comparator-based generic ranking, and PriorityQueue for severity-based service. The Q7 redesign improved front-insertion performance in this Colab run.

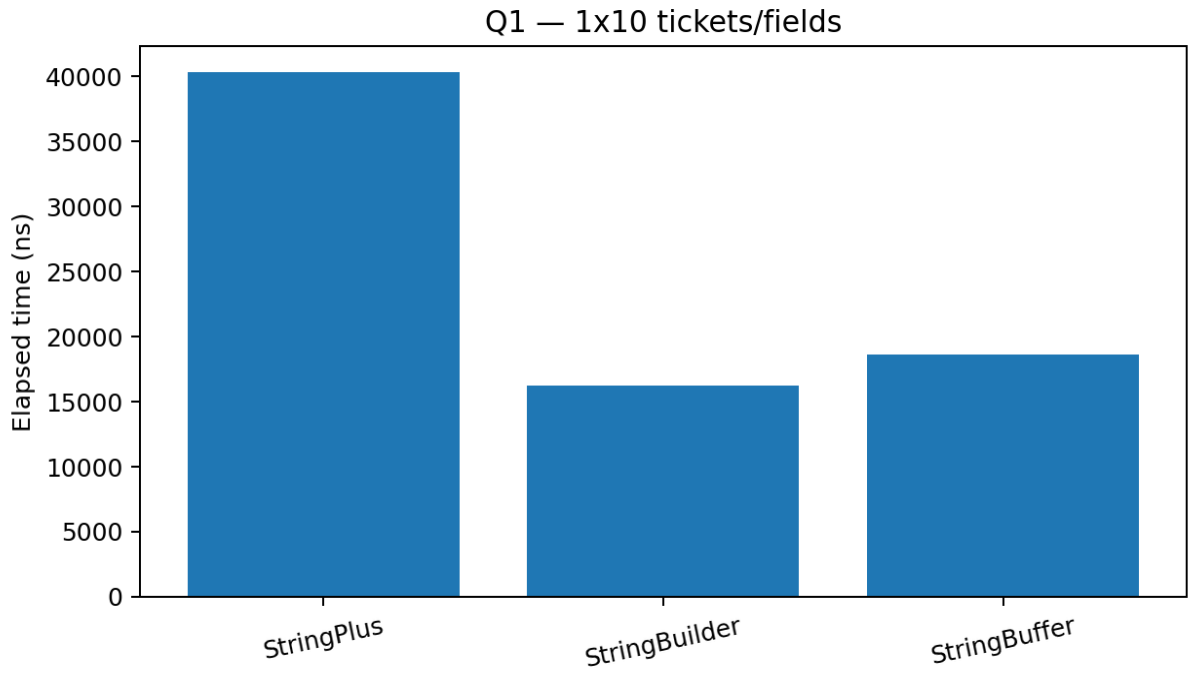
Submission note

The package includes source code, benchmark CSVs, graphs, and seven output evidence panels reconstructed from notebook output. These PNGs are not literal browser screenshots. For strict screenshot evidence, capture the actual Colab output cells. Replace the Q2 preview CSV with the full q2_measurements.csv downloaded from Colab. Create and push the GitHub repository using the README instructions; remote publication cannot be done without account authorization.

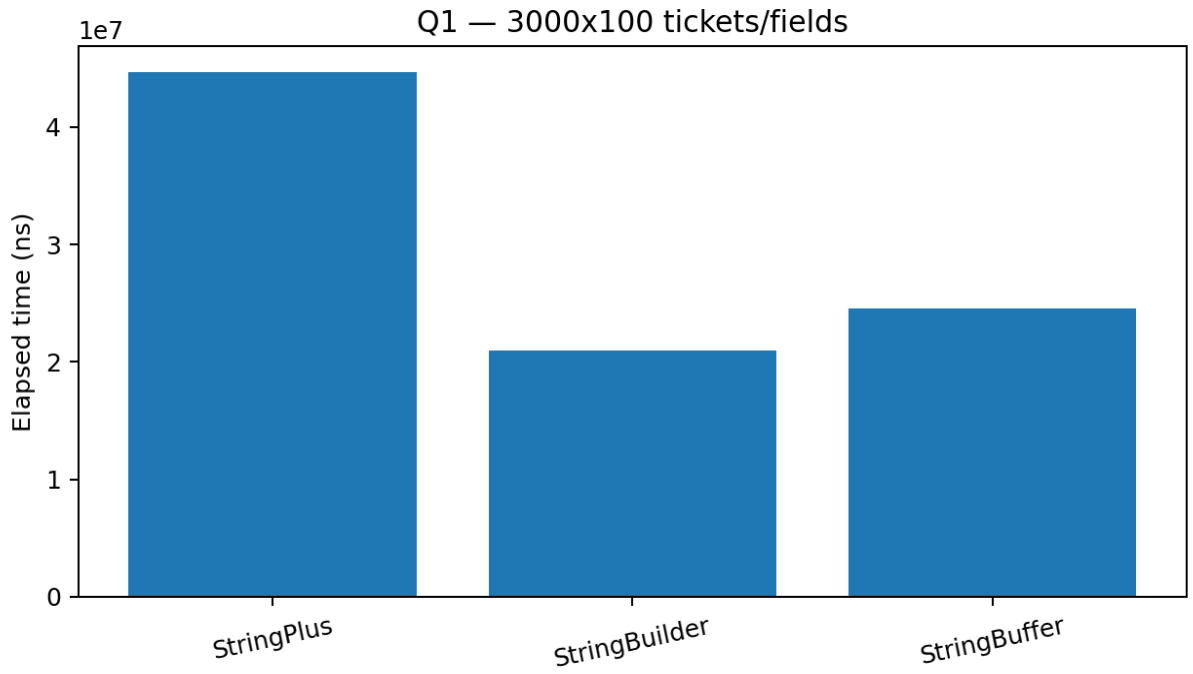
Q1 1000x50


Q1 — 1000x50 tickets/fields
1e7
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
Elapsed time (ns)
StringPlus
StringBuilder
StringBuffer

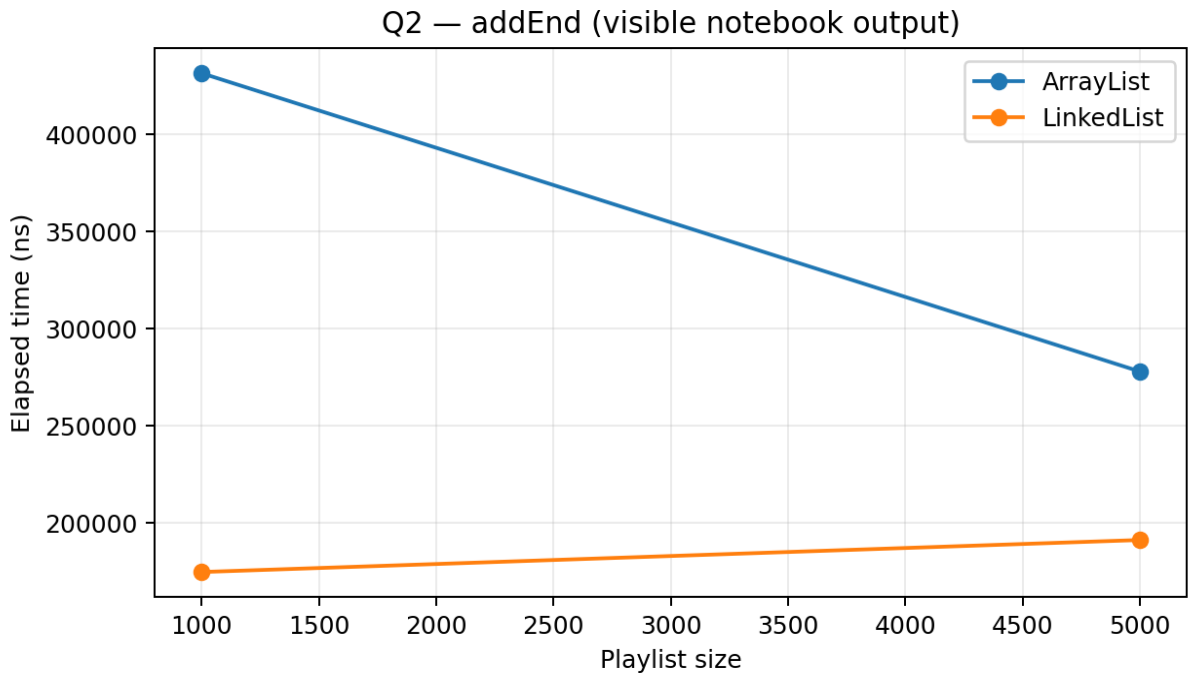
Q1 1x10


Q1 — 1x10 tickets/fields
Elapsed time (ns)
40000
35000
30000
25000
20000
15000
10000
5000
0
StringPlus
StringBuilder
StringBuffer

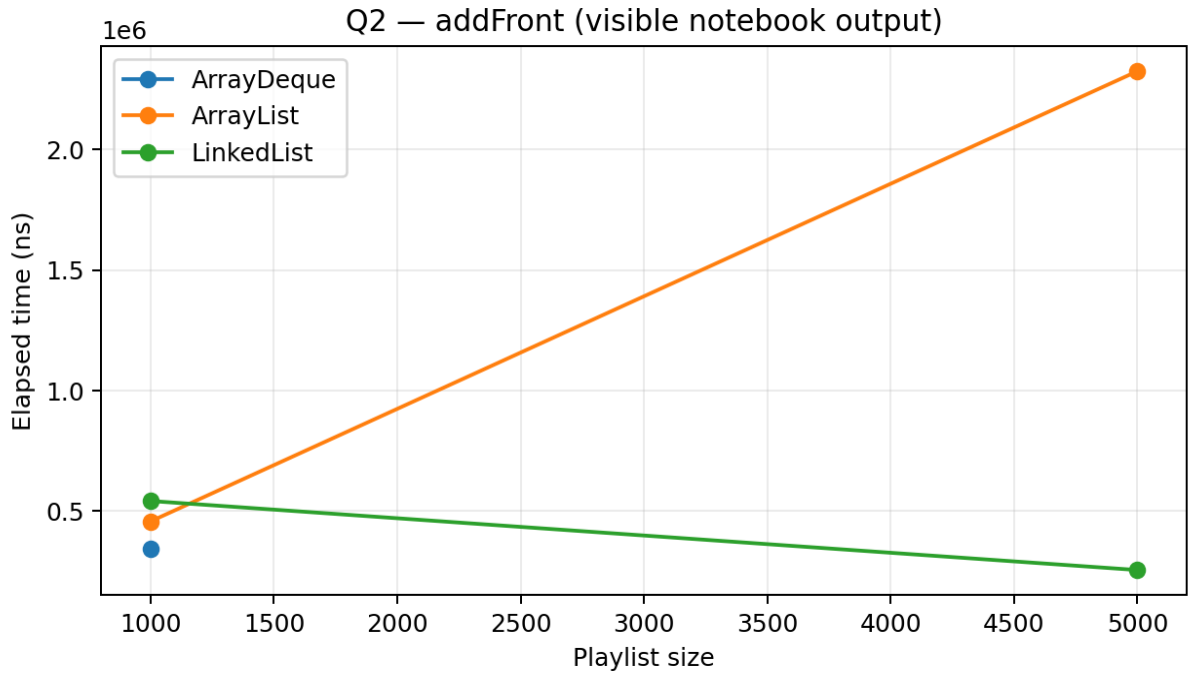
Q1 3000x100


Q1 — 3000x100 tickets/fields
1e7
Elapsed time (ns)
0
1
2
3
4
StringPlus
StringBuilder
StringBuffer

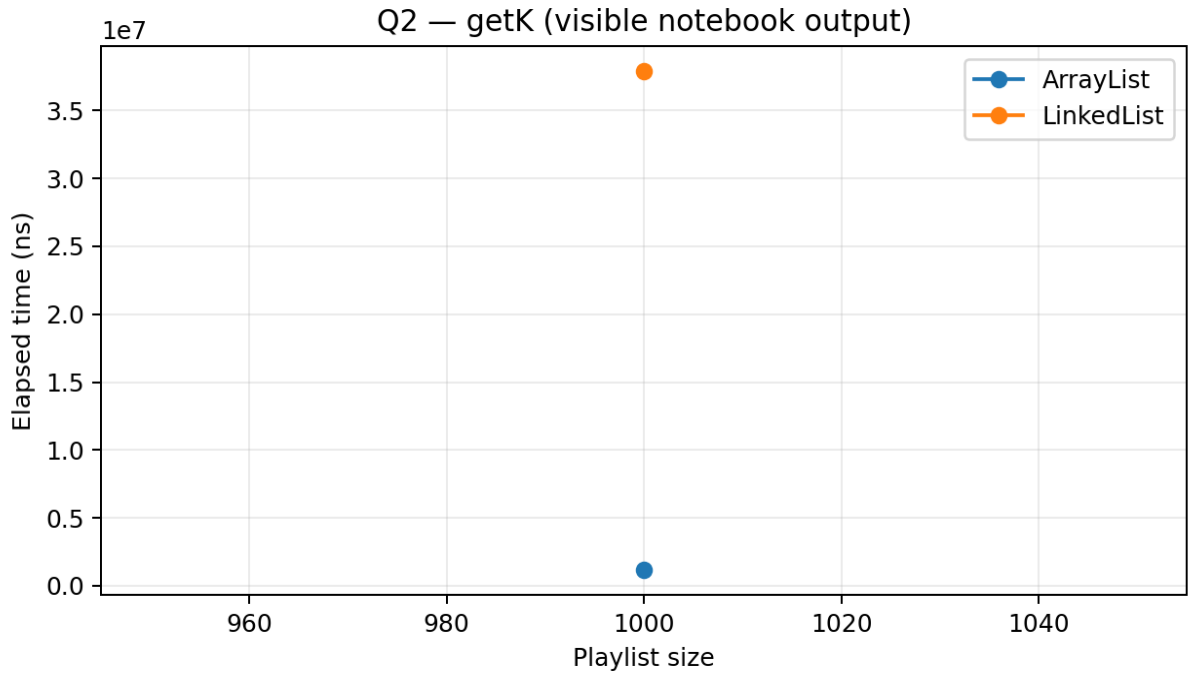
Q2 addEnd


Q2 — addEnd (visible notebook output)
ArrayList
LinkedList
Elapsed time (ns)
400000
350000
300000
250000
200000
1000
1500
2000
2500
3000
3500
4000
4500
5000
Playlist size

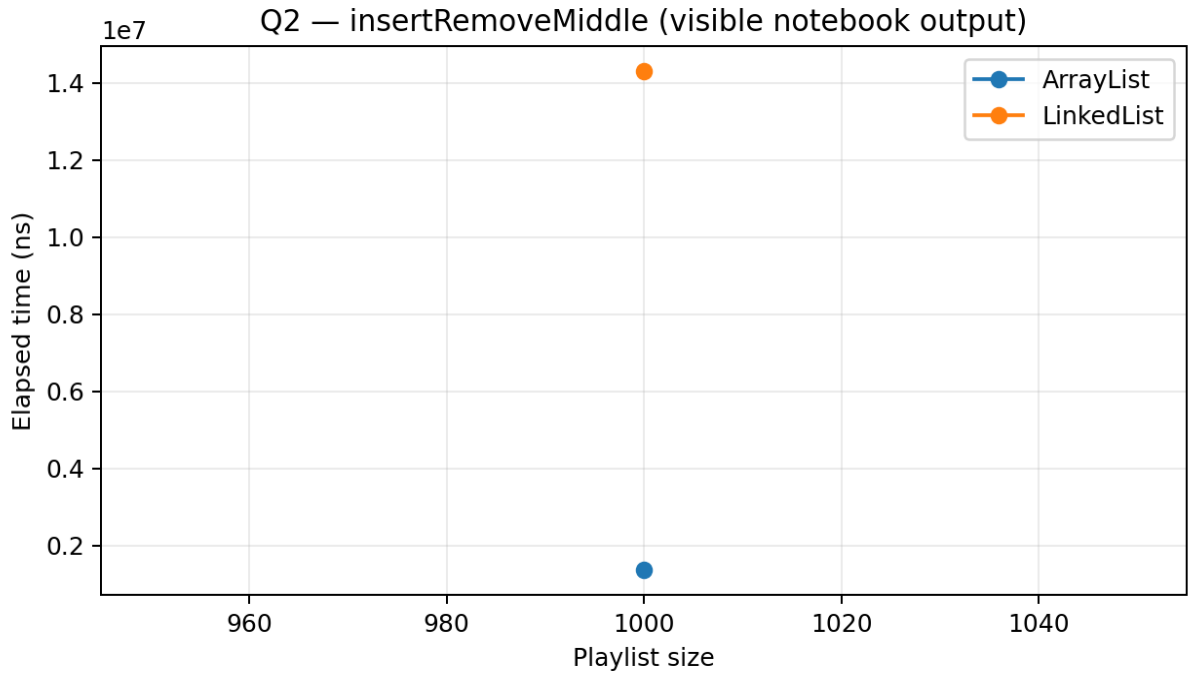
Q2 addFront


Q2 — addFront (visible notebook output)
1e6
ArrayDeque
ArrayList
LinkedList
Elapsed time (ns)
2.0
1.5
1.0
0.5
1000
1500
2000
2500
3000
3500
4000
4500
5000
Playlist size

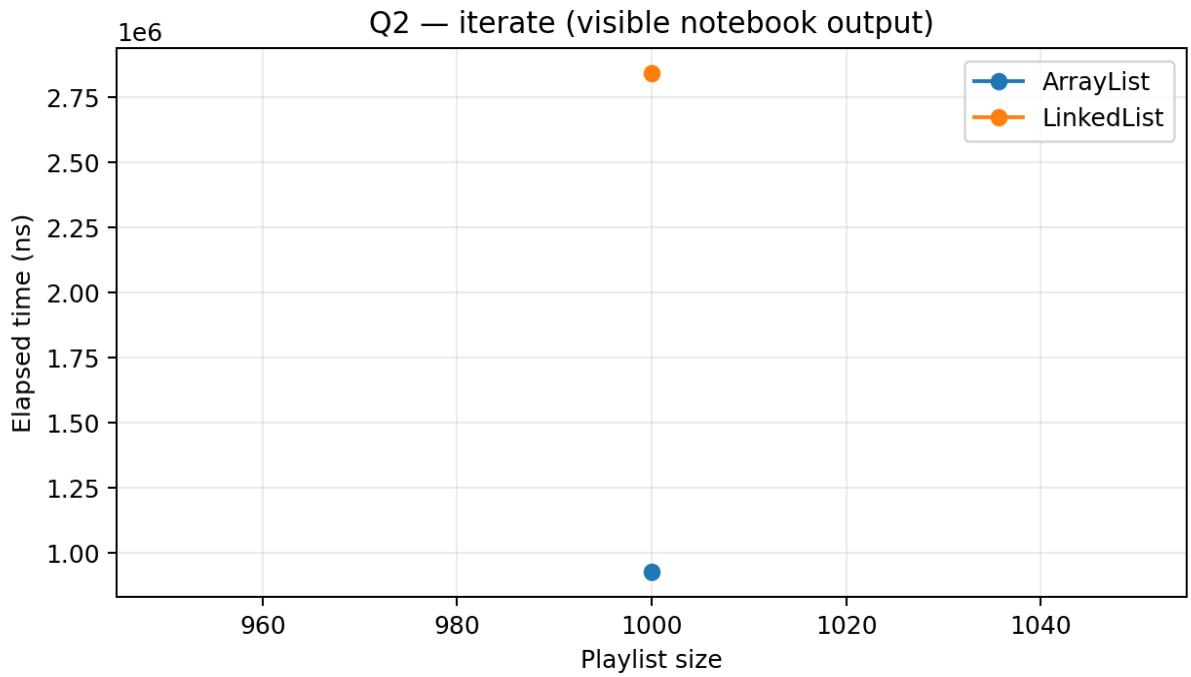
Q2 getK


Q2 — getK (visible notebook output)
1e7
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Elapsed time (ns)
960
980
1000
1020
1040
Playlist size
ArrayList
LinkedList

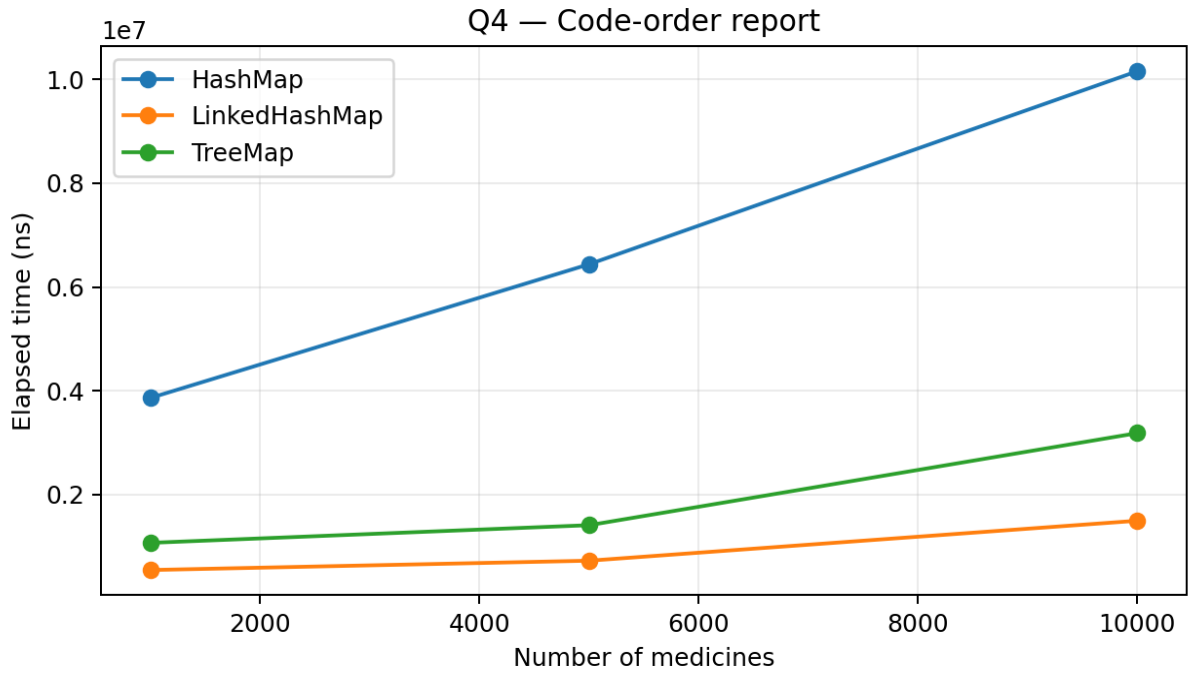
Q2 insertRemoveMiddle



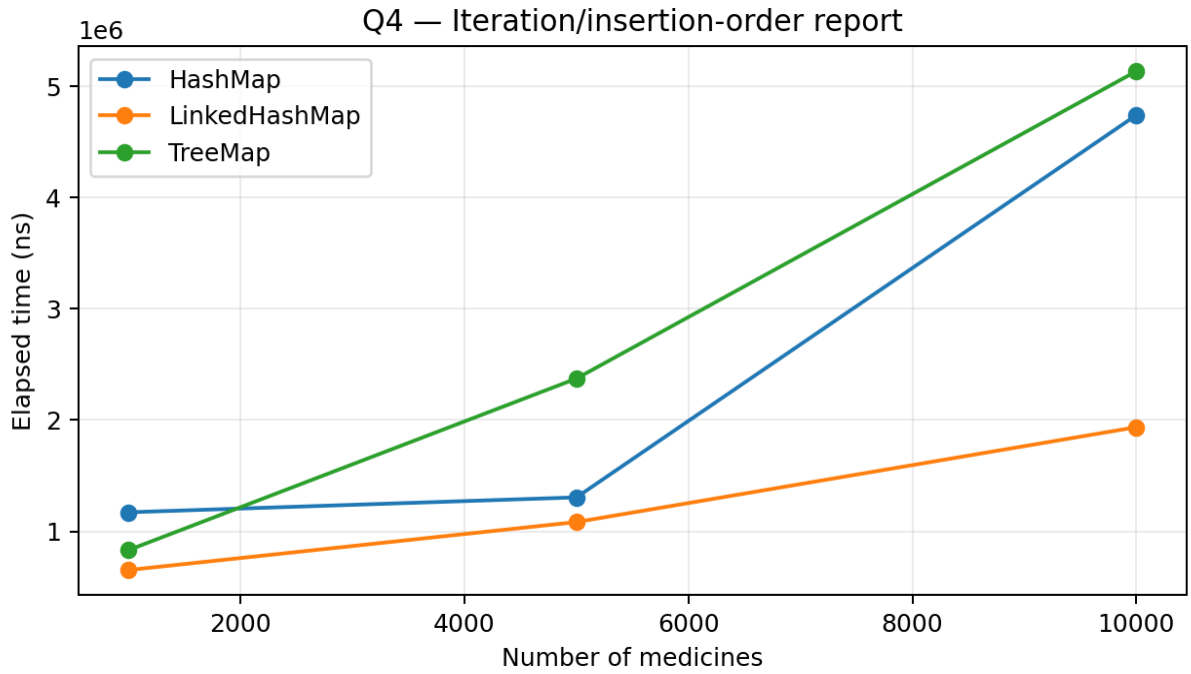
Q2 iterate


Q2 — iterate (visible notebook output)
1e6
2.75
2.50
2.25
2.00
1.75
1.50
1.25
1.00
Elapsed time (ns)
960
980
1000
1020
1040
Playlist size
ArrayList
LinkedList

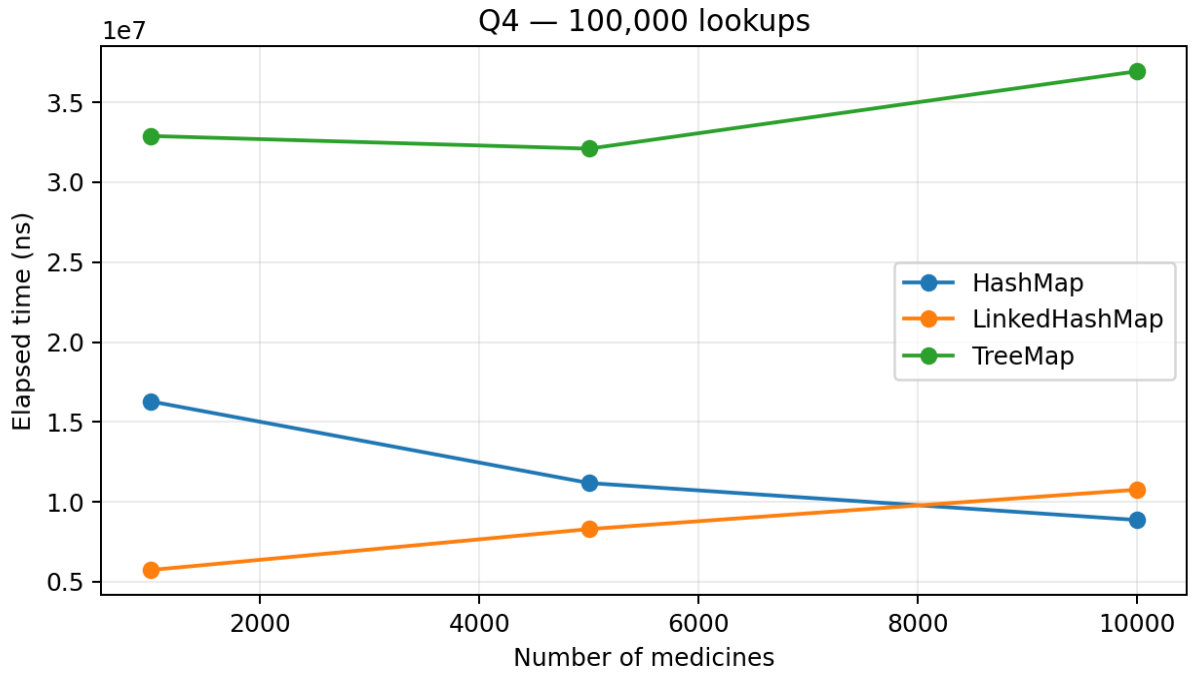
Q4 code report



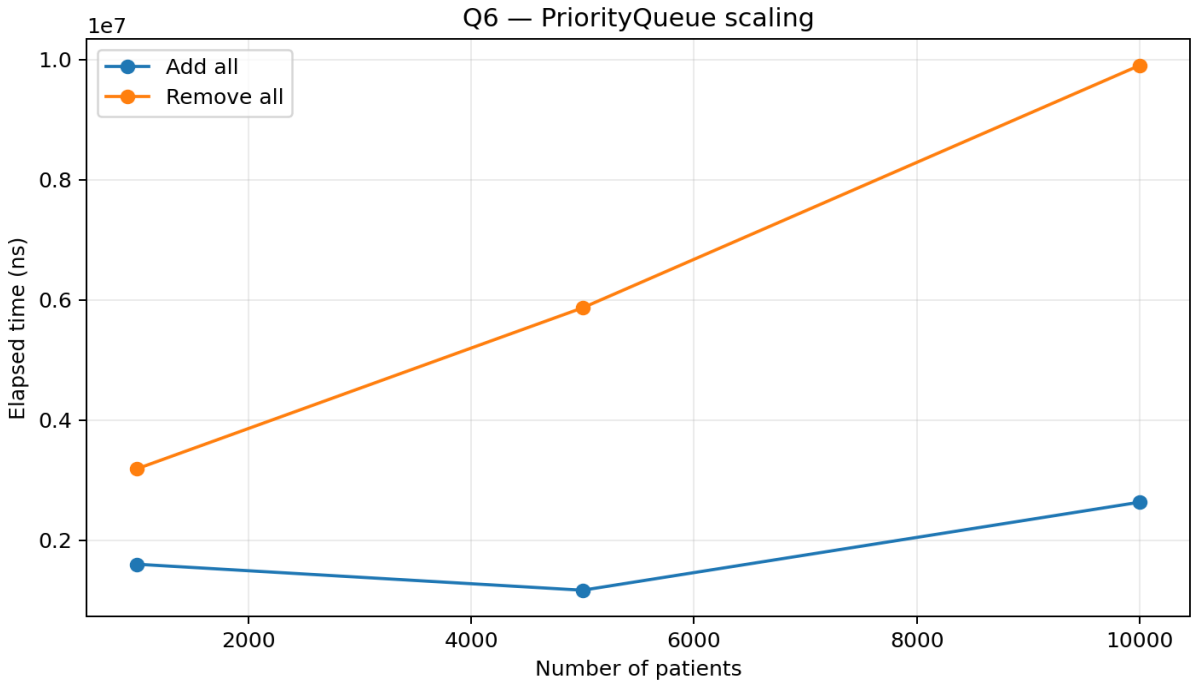
Q4 iteration



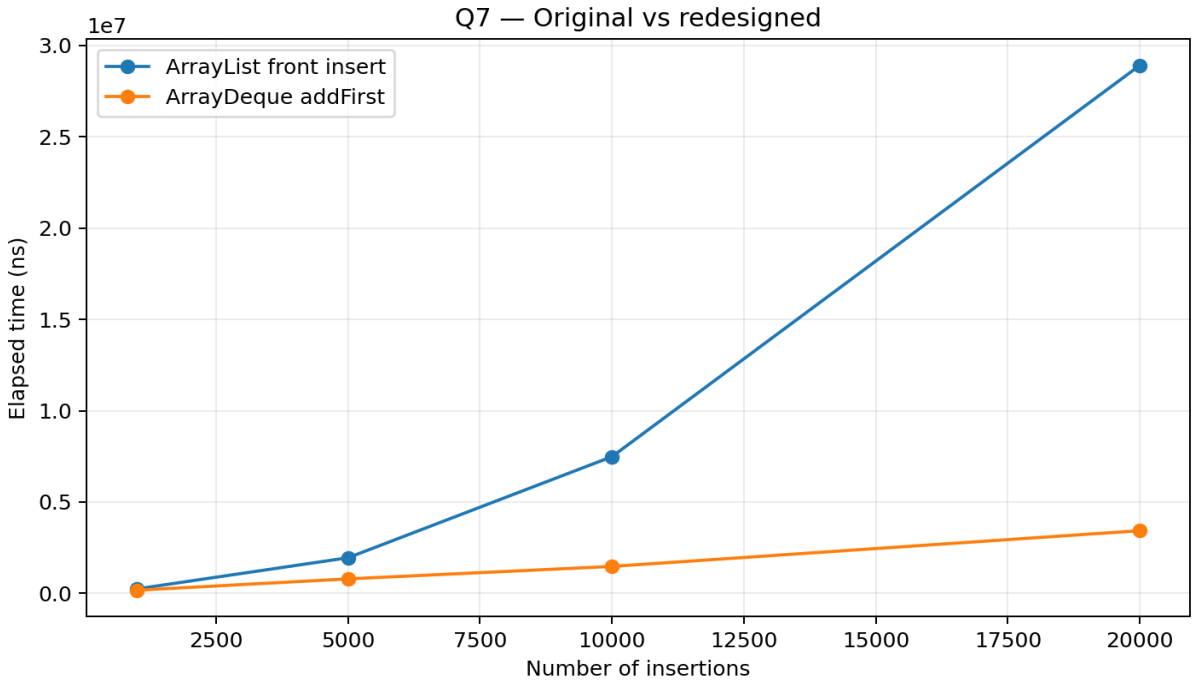
Q4 lookup


Q4 — 100,000 lookups
1e7
3.5
3.0
2.5
2.0
1.5
1.0
0.5
Elapsed time (ns)
2000
4000
6000
8000
10000
Number of medicines
HashMap
LinkedHashMap
TreeMap

Q6 priority queue



Q7 redesign


Q7 — Original vs redesigned
1e7
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Elapsed time (ns)
ArrayList front insert
ArrayDeque addFirst
2500
5000
7500
10000
12500
15000
17500
20000
Number of insertions